Vietnam National University, Ho Chi Minh City
Ho Chi Minh City University of Technology
Faculty of Computer Science and Engineering

DISCRETE STRUCTURE

The Traveling Salesman Problem

Class: C01007 Semester: 243
Instructor: Tran Tuan Anh
Tran Hong Tai

Student:

Full Name	**Student ID**
Le Hien Vinh	2453422

Ho Chi Minh City, 8-2025

Contents

1. **Acknowledgement** 2

2. **Introduction** 3

3. **Approach to Solving TSP** 4

   3.1. Exact Algorithms 4

      3.1.1. Brute Force Approach (Exhaustive Search) 4

      3.1.2. Dynamic Programming (Held-Karp Algorithm) 4

   3.2. Heuristic and Approximation Algorithms 5

      3.2.1. Nearest Neighbor Heuristic (Selected Approach) 6

      3.2.2. Greedy Edge-Adding Algorithm 7

      3.2.3. Christofides' Algorithm 8

   3.3. Optimization Techniques to Improve Solutions 9

      3.3.1. 2-opt Algorithm 9

4. **Combined Approach: Nearest Neighbor + 2-opt (Selected Approach)** 11

5. **Implementation of the Combined Approach** 11

   5.1. Code Overview 11

   5.2. Implementation in C++ 13

6. **Conclusion and Complexity Analysis** 16

1 Acknowledgement

First and foremost, I would like to express my sincere gratitude to the Ho Chi Minh City University of Technology for including the Discrete Structure course in the academic program. This subject provides a solid foundation for understanding fundamental mathematical concepts, helps me analyze the nature of discrete events, nurtures both critical thinking and practical problem-solving skills, and equips me with the necessary knowledge before advancing to more specialized subjects. Additionally, I deeply appreciates the university's commitment to providing the essential resources, facilities, and an enriching academic environment that allowed me to complete our project in Discrete Structure.

I would also like to extend my deepest thanks to Instructor Tran Tuan Anh and Instructor Tran Hong Tai for their guidance, dedication, and encouragement throughout the course. Your expertise and passion for teaching have been a constant source of inspiration, helping me understand complex concepts and develop a systematic approach to problem-solving.

2 Introduction

TSP Model

The travelling salesman problem, also known as the travelling salesperson problem (TSP), asks the following question:

"Given a list of cities and the distances between each pair of cities, what is the shortest possible route that visits each city exactly once and returns to the origin city?"

It is an NP-hard problem in combinatorial optimization, important in theoretical computer science and operations research-from Wikipedia

TSP is typically modeled as a Hamiltonian cycle on a weighted complete graph, where each vertex represents a city and each edge denotes the cost or distance between cities. Due to the factorial growth in the number of possible tours—specifically, $\frac{(n-1)!}{2}$ unique routes for n cities—the problem becomes computationally infeasible to solve exactly for large graphs.

As a result, heuristic and approximation algorithms are commonly employed to find near-optimal solutions within reasonable time constraints. These include methods such as Nearest Neighbor, Greedy, Christofides' algorithm, and 2-opt/3-opt local search techniques, which offer a trade-off between solution quality and execution speed.

Exercise: Path Calculation

Write function `Traveling()` to calculate the shortest way to go over all the vertices in the graph and go back to the starting vertex. (5 pts)

Input:

- The Graph
- Number of vertices
- Start vertex (A-Z then a-z)

Output:

- A string represents the shortest path over all the vertices and returns to the starting vertex. (i.e.: "A B C D E F G H A")

3 Approach to Solving TSP

3.1 Exact Algorithms

3.1.1 Brute Force Approach (Exhaustive Search)

In this approach, all possible permutations of the cities are generated, and the total travel cost is calculated for each permutation. The solution with the minimum cost is the optimal solution. This method guarantees an optimal solution but has an exponential time complexity of $O(n!)$, making it impractical for larger instances of TSP (e.g., 20+ cities).

Pseudocode

```
function BruteForceTSP(cities, distance_matrix):
    best_tour = null
    min_distance = infinity

    # Generate all permutations of cities
    for each permutation of cities:
        distance = calculateTotalDistance(permutation, distance_matrix)
        if distance < min_distance:
            min_distance = distance
            best_tour = permutation

    return best_tour
```

Advantages

- Guarantees the optimal solution.
- Simple to understand and implement.

Disadvantages

- Computationally expensive for large n due to the factorial growth of possible solutions.
- Not practical for more than 10-12 cities due to time constraints.

3.1.2 Dynamic Programming (Held-Karp Algorithm)

The Held-Karp algorithm is an exact solution for TSP that uses dynamic programming to reduce the time complexity to $O(n^2 2^n)$. It breaks the problem into subproblems, stores intermediate results, and combines them to obtain the final optimal tour. Although faster than brute force, it is still impractical for large instances due to its exponential growth in time and memory.

Pseudocode

```
function HeldKarpTSP(distance_matrix):
    n = number of cities
    dp = array of size 2^n x n initialized to infinity
    dp[1][0] = 0  # Starting city

    for subset_size from 2 to n:
        for each subset of cities of size subset_size:
            for each city in subset:
                for each previous_city in subset:
                    dp[subset][city] = min(dp[subset][city],
                    dp[subset - {city}][previous_city]
                    + distance(previous_city, city))

    min_tour_length = infinity
    for city in range(1, n):
        min_tour_length = min(min_tour_length, dp[all_cities][city]
        + distance(city, start))

    return min_tour_length
```

Advantages

- Guarantees the optimal solution.
- More efficient than brute force, especially for smaller to moderate-sized TSP instances.

Disadvantages

- Still has exponential time complexity, though reduced.
- Memory-intensive and impractical for large instances of TSP.

3.1.3 Why We Didn't Choose Exact Algorithms

Although exact algorithms like Brute Force and Dynamic Programming guarantee an optimal solution, their time complexity grows exponentially with the number of cities. As the number of cities increases, the computational cost becomes prohibitively high. For real-world instances with many cities (e.g., 50 or more), exact algorithms become infeasible. Thus, they are useful for small instances but impractical for large-scale TSP.

3.2 Heuristic and Approximation Algorithms

Heuristic and approximation algorithms provide near-optimal solutions for the Travelling Salesman Problem (TSP) with significantly lower computational cost than

exact algorithms. These methods are practical for larger instances where exact solutions are infeasible.

3.2.1 Nearest Neighbor Heuristic (Selected Approach)

Idea: Start at a given city and repeatedly visit the nearest unvisited city until all cities are visited, then return to the starting city. This produces a reasonable tour quickly but does not guarantee optimality.

Steps

1. Mark the starting city as visited.
2. While unvisited cities remain, move to the nearest unvisited city and mark it visited.
3. Return to the starting city after visiting all cities.
4. Output the ordered tour of all cities.

Pseudocode

```
function NearestNeighborTSP(graph, n, start):
    visited = [False] * n
    path = [start]
    visited[start] = True
    current = start

    for i = 1 to n-1:
        nearest = None
        min_distance = infinity
        for city = 0 to n-1:
            if not visited[city] and graph[current][city] < min_distance:
                nearest = city
                min_distance = graph[current][city]

        path.append(nearest)
        visited[nearest] = True
        current = nearest

    path.append(start)  # return to the starting city
    return path
```

Time Complexity

$$O(n^2), \quad n = \text{number of cities}$$

Advantages

- Simple, fast, and easy to implement.
- Provides reasonable solutions for small to medium instances.

Disadvantages

- Does not guarantee the optimal solution.
- Can get trapped in local optima; performance may degrade for large or dense graphs.

3.2.2 Greedy Edge-Adding Algorithm

Idea: Build a tour by repeatedly adding the shortest available edge, while avoiding premature cycles and keeping city degrees ¡=2, until a complete tour is formed.

Pseudocode

```
function GreedyEdgeAdding(graph):
    edges = sortEdgesByWeight(graph)
    # sort all edges in ascending order

    tour = []
    DSU = initializeDisjointSet(graph)
    # for cycle detection

    degree = [0] * number_of_cities

    for (u, v) in edges:
        if degree[u] < 2 and degree[v] < 2
        and not createsCycle(u, v, DSU):
            tour.append((u, v))
            union(u, v, DSU)
        if len(tour) == number_of_cities:
            break

    return tour  # returns a list of edges forming the tour
```

Time Complexity

$$O(E \log E), \quad E = \text{number of edges}$$

Advantages

- Efficient for sparse or medium-sized graphs.
- Fast initialization for other optimization techniques.

Disadvantages

- May block better edges if cycles form too early.
- Not guaranteed to produce an optimal tour.

3.2.3 Christofides' Algorithm

Idea

- **Minimum Spanning Tree (MST)**: Compute the MST of the cities using Prim's or Kruskal's algorithm.
- **Find Odd Degree Cities**: Identify cities in the MST with an odd degree.
- **Perfect Matching**: Find the minimum weight perfect matching for the odd-degree cities.
- **Combine MST and Matching**: Add the matching edges to the MST, ensuring all cities have even degrees.
- **Eulerian Circuit and Shortcutting**: Find an Eulerian circuit, then shortcut repeated cities to form a Hamiltonian cycle.
- **Output**: Return the Hamiltonian cycle as the TSP solution.

Pseudocode

```
function Christofides(graph):
    MST = minimumSpanningTree(graph)
    odd_cities = findOddDegreeCities(MST)
    matching = minimumWeightMatching(odd_cities, graph)
    combined_graph = MST + matching
    eulerian_circuit = findEulerianCircuit(combined_graph)
    hamiltonian_circuit = shortcutEulerianCircuit(eulerian_circuit)
    return hamiltonian_circuit
```

Time Complexity

The time complexity of Christofides' algorithm is:

$$\mathcal{O}(V^3)$$

Where V is the number of cities. The step of finding the minimum weight perfect matching dominates the complexity, which is $O(V^3)$.

Advantages

- **Approximation Guarantee**: Guarantees a solution within 1.5 times the optimal solution.

- **Effective for Metric TSP**: Works well for metric TSP where triangle inequality holds.
- **Efficient for Large Instances**: Provides a good balance between solution quality and computation time.

Disadvantages

- **Not Optimal**: The algorithm provides an approximation, not the optimal solution.
- **Expensive Matching**: The matching step has a high time complexity of $O(V^3)$.
- **Requires Metric Graph**: Only applicable to metric TSP, where triangle inequality holds.

3.2.4 Why We Chose Nearest Neighbor

The Nearest Neighbor (NN) heuristic was selected for its simplicity, speed, and flexibility. It quickly produces a reasonable tour, making it suitable for larger graphs where exact solutions are impractical.

Unlike Christofides' Algorithm, NN works on any graph without requiring the triangle inequality. Compared to the Greedy Edge-Adding Algorithm, NN is more predictable and less prone to subtour conflicts.

Using NN for initial construction and 2-opt for local optimization provides fast, high-quality tours while keeping the implementation simple and efficient.

3.3 Optimization Techniques to Improve Solutions

3.3.1 2-opt Algorithm (Selected Approach)

Idea: Improve a given tour (e.g., from Nearest Neighbor) by iteratively swapping two edges if the swap shortens the total tour length. The goal is to remove "crossed" edges and reduce overall distance.

Steps

1. Take the current tour.
2. For every pair of non-adjacent edges, check if swapping them reduces the tour length.
3. If an improvement is found, perform the swap.
4. Repeat until no further improvement is possible.

Pseudocode

```
function TwoOpt(tour, distance_matrix):
    improvement = true
    n = length(tour)
```

```
while improvement:
    improvement = false
    for i = 1 to n-2:
        for j = i+1 to n-1:
            # Calculate the change in distance if we swap edges
            delta = (distance_matrix[tour[i-1]][tour[j]]
            + distance_matrix[tour[i]][tour[j+1]])
            - (distance_matrix[tour[i-1]][tour[i]]
            + distance_matrix[tour[j]][tour[j+1]])

            if delta < 0:
                # Perform the 2-opt swap
                reverse tour[i..j]
                improvement = true

return tour
```

Time Complexity

$$O(n^2) \text{ per iteration}, \quad n = \text{number of cities}$$

Advantages

- Simple to implement.
- Significantly improves initial tours.
- Provides a good approximation to the optimal solution.

Disadvantages

- Only performs local optimization; may not reach global optimum.
- May require multiple iterations to converge.

Why We Chose 2-opt

- Refines the Nearest Neighbor solution efficiently.
- Eliminates path crossings for a shorter tour.
- Achieves high-quality approximations without the cost of exact methods.
- Combines speed and effectiveness, making it practical for large TSP instances.

4 Combined Approach: Nearest Neighbor + 2-opt (Selected Approach)

Rationale: Combining Nearest Neighbor (NN) with 2-opt leverages the strengths of both methods:

- **Efficiency:** NN quickly generates an initial tour in polynomial time, suitable for large graphs.
- **Improvement:** 2-opt refines the tour by eliminating edge crossings, reducing total distance.
- **Scalability:** This approach handles larger TSP instances, unlike exact algorithms which are computationally expensive.

Steps of the Combined Approach

1. Construct an initial tour using the Nearest Neighbor heuristic.
2. Apply 2-opt to iteratively swap edges and remove inefficient crossings.
3. Repeat 2-opt until no further improvement is possible.
4. Return the optimized tour.

Implementation in C++:
Conclusion: This combined approach balances speed and solution quality, producing near-optimal tours efficiently for practical TSP instances.

5 Implementation of the Combined Approach

The following C++ implementation demonstrates the Combined Approach using Nearest Neighbor and 2-opt for solving the Traveling Salesman Problem (TSP).

Code Overview

- **vertex_to_index(char vertex_label):** Converts a vertex character (A-Z, a-z) into its corresponding index (0-51). This allows us to use characters as input while internally working with integers.
- **index_to_vertex(int index):** Converts an index back into the vertex character. This is useful for displaying the final tour in the required format.
- **is_edge_valid(int u, int v, int graph[60][60]):** Checks whether an edge exists between two vertices in the graph. An edge is valid if the weight is positive (non-zero).
- **tour_cost(const vector¡int¿& tour, int graph[60][60]):** Calculates the total distance of a given tour. Returns -1 if the tour is invalid (e.g., missing edges).

- **nearest_neighbor(int graph[60][60], int vertices, int start):** Constructs an initial tour using the Nearest Neighbor heuristic:
  1. Starts from the given vertex.
  2. At each step, chooses the nearest unvisited neighbor.
  3. Returns the initial tour as a vector of indices.
- **two_opt(vector¡int¿& tour, int graph[60][60]):** Optimizes a tour using the 2-opt method:
  1. Iteratively swaps edges to remove crossings.
  2. Repeats until no improvement is possible or a maximum number of iterations is reached.
  3. Updates the tour in-place.
- **Traveling(int graph[60][60], int vertices, char start_char):** Combines Nearest Neighbor and 2-opt to produce the optimized tour:
  1. Converts the start vertex character into an index.
  2. Generates multiple tours using Nearest Neighbor from each vertex.
  3. Optimizes each tour with 2-opt.
  4. Selects the tour with the minimal cost that includes the start vertex.
  5. Returns the tour as a string of vertex characters, including a return to the start.

Code Listing

```
#include "tsm.h"
#include <iostream>
const long long INF = 1000000000000LL;

int vertex_to_index(char vertex_label) { ... }
char index_to_vertex(int index) { ... }
bool is_edge_valid(int u, int v, int graph[60][60]) { ... }
long long tour_cost(const vector<int>& tour, int graph[60][60]) { ... }
vector<int> nearest_neighbor(int graph[60][60], int vertices, int start)
{ ... }
void two_opt(vector<int>& tour, int graph[60][60]) { ... }
string Traveling(int graph[60][60], int vertices, char start_char)
{ ... }
```

Explanation of Workflow

1. Start by converting the input start vertex character into an index.
2. Construct an initial tour using `nearest_neighbor`.

3. Optimize the tour using `two_opt` to eliminate inefficient edges.
4. Repeat for each possible starting vertex to find the best tour.
5. Rotate the tour so it starts from the desired vertex and return it as a string.

This implementation efficiently produces near-optimal solutions for TSP instances using a combination of fast heuristics and local optimization.

Implementation in C++

Code Listing

```
#include "tsm.h"
#include <iostream>
const long long INF = 1000000000000LL;

// Converts vertex label (A-Z, a-z) to an integer index (0-51)
int vertex_to_index(char vertex_label) {
    if (vertex_label >= 'A' && vertex_label <= 'Z')
    return vertex_label - 'A';
    if (vertex_label >= 'a' && vertex_label <= 'z')
    return 26 + (vertex_label - 'a');
    return -1;
}

// Converts index back to vertex label
char index_to_vertex(int index) {
    if (index < 26) return 'A' + index;
    return 'a' + (index - 26);
}

// Checks if an edge exists between two vertices
bool is_edge_valid(int u, int v, int graph[60][60]) {
    return graph[u][v] > 0;
}

// Calculates the total cost of a tour, returns -1 if invalid
long long tour_cost(const vector<int>& tour, int graph[60][60]) {
    long long cost = 0;
    int n = tour.size();
    if (n < 2) return -1;
    for (int i = 0; i < n - 1; ++i) {
        if (!is_edge_valid(tour[i], tour[i+1], graph)) return -1;
        cost += graph[tour[i]][tour[i+1]];
    }
```

```
    if (!is_edge_valid(tour.back(), tour[0], graph)) return -1;
    cost += graph[tour.back()][tour[0]];
    return cost;
}

// Constructs an initial tour using Nearest Neighbor heuristic
vector<int> nearest_neighbor(int graph[60][60], int vertices, int start){
    vector<int> tour;
    vector<bool> visited(vertices, false);
    visited[start] = true;
    tour.push_back(start);
    int current = start;
    for (int step = 1; step < vertices; ++step) {
        int best = -1;
        long long best_dist = INF;
        for (int i = 0; i < vertices; ++i) {
            if (!visited[i] && is_edge_valid(current, i, graph)
            && graph[current][i] < best_dist) {
                best = i;
                best_dist = graph[current][i];
            }
        }
        if (best == -1) return {};
        visited[best] = true;
        tour.push_back(best);
        current = best;
    }
    if (!is_edge_valid(tour.back(), start, graph)) return {};
    return tour;
}

// Optimizes a tour using 2-opt to remove inefficient edges
void two_opt(vector<int>& tour, int graph[60][60]) {
    int n = tour.size();
    if (n <= 3) return;
    bool improved = true;
    int iteration = 0;
    const int MAX_ITER = 1000;
    long long current_cost = tour_cost(tour, graph);
    if (current_cost == -1) return;
    while (improved && iteration++ < MAX_ITER) {
        improved = false;
        for (int i = 1; i < n - 1; ++i) {
            for (int k = i + 1; k < n; ++k) {
```

```
                vector<int> candidate = tour;
                reverse(candidate.begin() + i, candidate.begin()
                + k + 1);
                long long new_cost = tour_cost(candidate, graph);
                if (new_cost != -1 && new_cost < current_cost) {
                    tour.swap(candidate);
                    current_cost = new_cost;
                    improved = true;
                    break;
                }
            }
            if (improved) break;
        }
    }
}

// Combines Nearest Neighbor and 2-opt to produce the optimized tour
string Traveling(int graph[60][60], int vertices, char start_char) {
    int start = vertex_to_index(start_char);
    vector<int> best_tour;
    long long best_cost = INF;
    for (int s = 0; s < vertices; ++s) {
        vector<int> tour = nearest_neighbor(graph, vertices, s);
        if (tour.size() != vertices) continue;
        two_opt(tour, graph);
        long long cost = tour_cost(tour, graph);
        if (cost != -1 && cost < best_cost) {
            best_cost = cost;
            best_tour = tour;
        }
    }
    if (best_tour.empty()) return "";
    auto it = find(best_tour.begin(), best_tour.end(), start);
    if (it == best_tour.end()) return "";
    rotate(best_tour.begin(), it, best_tour.end());
    string result;
    for (int v : best_tour) {
        if (!result.empty()) result += " ";
        result += index_to_vertex(v);
    }
    result += " ";
    result += index_to_vertex(best_tour[0]);
    return result;
    }
```

6 Conclusion and Complexity Analysis

Conclusion

The implemented Combined Approach using Nearest Neighbor (NN) and 2-opt provides an effective method for solving practical instances of the Traveling Salesman Problem (TSP). The Nearest Neighbor heuristic quickly constructs an initial feasible tour, while the 2-opt optimization iteratively refines it by eliminating inefficient edge crossings.

This approach balances computational efficiency and solution quality, producing near-optimal tours for graphs that are too large for exact algorithms. While it may not guarantee a globally optimal solution, it is highly suitable for applications where speed and scalability are important.

Complexity Analysis

- **Nearest Neighbor Heuristic:** For a graph with n vertices, NN requires checking all unvisited vertices at each step. The time complexity is:

  $$O(n^2)$$

  since for each of the n steps we may examine up to n vertices.

- **2-opt Optimization:** In the worst case, 2-opt considers all pairs of edges in the tour for potential swapping. For a tour with n vertices, there are $O(n^2)$ possible swaps per iteration. If the algorithm runs for k iterations, the worst-case complexity is:

  $$O(k \cdot n^2)$$

  In practice, k is usually much smaller than n, especially with the maximum iteration limit, making the algorithm efficient for moderately large graphs.

- **Combined Approach:** Since the algorithm can start NN from all possible starting vertices (up to n), the overall worst-case complexity becomes:

  $$O(n \cdot (n^2 + k \cdot n^2)) = O(k \cdot n^3)$$

  Despite this cubic worst-case bound, the approach performs well in practical scenarios and is significantly faster than exact algorithms like dynamic programming or branch-and-bound.

Summary: The Combined Approach offers a practical trade-off between speed and accuracy, making it a preferred choice for medium to large TSP instances where exact solutions